# Week 4 · Lesson 4
# Baseline Machine Translation

**Translate, measure, and evaluate | Approximately 60 minutes**

You will use a pretrained English-to-French translation pipeline, measure batched inference speed, compute corpus BLEU against French references, and inspect low-scoring translations. You will save a baseline for later comparison; you will not train or fine-tune a model in this exercise.

**Create your notebook.** In Google Colab, select **File → New notebook** and name it **week4_lesson4_mt_baseline.ipynb**. Add a code cell for each block in Steps 1-6, in order, then a markdown cell for Step 7. No starter notebook is supplied. Start with a fresh runtime.

**Runtime:** Internet access is required for installation and the first model download. A GPU is optional and helpful; the code also runs on CPU. Allow the first download to finish before measuring translation speed.

## 1. Set up the environment

Use **%pip** inside Colab. SentencePiece and Sacremoses support the Marian tokenizer. **time** and **json** are standard-library modules and must not be installed with pip. The fixed Transformers 4.x version retains the translation pipeline used in this activity. [1]

```
%pip install -q "transformers==4.57.6" "torch>=2.6,<3" \
    "sacrebleu==2.5.1" sentencepiece sacremoses pandas

import os
os.environ["USE_TF"] = "0"
os.environ["USE_FLAX"] = "0"
import json
import time
from pathlib import Path
import torch
import pandas as pd
import transformers
import sacrebleu
from sacrebleu.metrics import BLEU
from IPython.display import display
from transformers import pipeline

DEVICE = 0 if torch.cuda.is_available() else -1
BATCH_SIZE = 4
torch.set_num_threads(2)
print("Device:", "GPU" if DEVICE == 0 else "CPU")
print("Versions:", torch.__version__, transformers.__version__,
      sacrebleu.__version__)
```

## 2. Load and test the English-to-French pipeline

```
MODEL_ID = "Helsinki-NLP/opus-mt-en-fr"
translator = pipeline(
    "translation_en_to_fr", model=MODEL_ID,
    framework="pt", device=DEVICE, dtype=torch.float32
)
GENERATION = dict(num_beams=4, do_sample=False, max_new_tokens=128)
print("Checkpoint:", MODEL_ID)
print("Decoding:", GENERATION)
print(translator("The weather is nice today.", **GENERATION))
```

The pipeline returns a list of dictionaries. The translated sentence is the string stored under **translation_text**. Beam search is fixed at four beams with sampling disabled. Keep these decoding settings unchanged when you compare baseline runs.

# Use aligned inputs and time translation

## 3. Prepare your parallel practice set

Use the complete 20-pair set below. Each English source has one corresponding French reference in the same position. These are illustrative teaching examples, not a WMT test set or a validated benchmark. Do not replace the references with translations generated by the model.

```
# Fixed practice pairs: (English source, French reference).
pairs = [
    ("The weather is nice today.", "Il fait beau aujourd'hui."),
    ("I love reading science fiction.", "J'adore lire de la science-fiction."),
    ("The train leaves at eight.", "Le train part à huit heures."),
    ("Please close the window.", "Veuillez fermer la fenêtre."),
    ("We booked a room for two nights.", "Nous avons réservé une chambre pour deux nuits."),
    ("The meeting starts on Monday.", "La réunion commence lundi."),
    ("My sister lives in Paris.", "Ma sœur habite à Paris."),
    ("They are learning French.", "Ils apprennent le français."),
    ("This book is more expensive.", "Ce livre est plus cher."),
    ("I have been waiting for two hours.", "J'attends depuis deux heures."),
    ("She did not receive the message.", "Elle n'a pas reçu le message."),
    ("Could you send me the invoice?", "Pourriez-vous m'envoyer la facture ?"),
    ("The bank raised interest rates.", "La banque a relevé les taux d'intérêt."),
    ("She sat on the bank of the river.", "Elle s'est assise sur la rive du fleuve."),
    ("We need to break the ice.", "Nous devons détendre l'atmosphère."),
    ("Please back up the files.", "Veuillez sauvegarder les fichiers."),
    ("The package arrived yesterday.", "Le colis est arrivé hier."),
    ("If it rains, we will stay home.", "S'il pleut, nous resterons à la maison."),
    ("The restaurant is next to the station.", "Le restaurant est à côté de la gare."),
    ("Thank you for your help.", "Merci pour votre aide."),
]
sources = [en for en, fr in pairs]
references = [fr for en, fr in pairs]
assert len(pairs) == 20 and all(en and fr for en, fr in pairs)
ids = translator.tokenizer(
    sources, truncation=False, verbose=False
)["input_ids"]
limit = translator.model.config.max_position_embeddings
assert max(map(len, ids)) <= limit, "Shorten long source texts."
print("Pairs:", len(pairs), "Source token limit:", limit)
```

**Keep the inputs fixed.** Preserve punctuation, accents, and sentence order. The assertion prevents unnoticed source truncation. The original WMT16 **ro-en** example is unsuitable: it is Romanian-English, not English-French. This self-contained set avoids that mismatch.

## 4. Run batch translation and measure speed

```
def synchronize():
    if DEVICE == 0:
        torch.cuda.synchronize()

kwargs = dict(batch_size=BATCH_SIZE, num_workers=0, **GENERATION)
translator(sources[:BATCH_SIZE], **kwargs)  # Warm-up; not timed.
synchronize()
start = time.perf_counter()
outputs = translator(sources, **kwargs)
synchronize()
elapsed = time.perf_counter() - start
predictions = [item["translation_text"] for item in outputs]
assert len(predictions) == len(sources)
assert all(isinstance(p, str) and p.strip() for p in predictions)
print(f"Batch size: {BATCH_SIZE}; total time: {elapsed:.2f} s")
print(f"Throughput: {len(sources) / elapsed:.2f} sentences/s")
print(f"Amortized time: {1000 * elapsed / len(sources):.2f} ms/sentence")
```

The warmed-up measurement includes tokenization, generation, and decoding, but excludes installation, loading, and warm-up. Dividing time by sentence count gives **amortized time**, not isolated-request latency. Report the device, batch size, and decoding settings with the result. These short inputs use a 128-token output cap; inspect any incomplete translation before deciding whether to increase it.

# Evaluate the baseline and inspect examples

## 5. Compute corpus BLEU, then inspect sentence diagnostics

Pass **translated strings** to SacreBLEU, not pipeline dictionaries. For one reference per sentence, the reference structure is a list containing the full reference list: **[references]**. Use the returned text directly; SacreBLEU applies its own evaluation tokenization. [2]

```
# One reference stream, with one reference per source sentence.
refs = [references]
bleu_metric = BLEU(tokenize="13a", lowercase=False,
                   smooth_method="exp", effective_order=False)
bleu = bleu_metric.corpus_score(predictions, refs)
signature = str(bleu_metric.get_signature())
print(f"Baseline corpus BLEU: {bleu.score:.2f} / 100")
print("Full result:", bleu)
print("SacreBLEU signature:", signature)
```

BLEU measures word-sequence overlap with references and includes a brevity penalty. Higher is better under identical evaluation conditions. The score is on a 0-100 scale; it is **not percentage accuracy**. Retain the printed signature so that tokenization, case handling, smoothing, and version are documented. [2]

**Add a second code cell for inspection.** Corpus BLEU does not tell you which individual sentences performed poorly. Compute sentence scores separately and display the three lowest-scoring cases.

```
# Sentence BLEU is a diagnostic, not the headline corpus result.
sentence_metric = BLEU(tokenize="13a", lowercase=False,
                       smooth_method="exp", effective_order=True)
sentence_scores = [
    sentence_metric.sentence_score(pred, [ref]).score
    for pred, ref in zip(predictions, references)
]
results = pd.DataFrame({
    "id": range(1, len(pairs) + 1), "source": sources,
    "reference": references, "prediction": predictions,
    "sentence_bleu": sentence_scores
})
display(results.head())
print("Three lowest-scoring sentences for manual review:")
for row in results.nsmallest(3, "sentence_bleu").itertuples():
    print(f"\nID {row.id}; sentence BLEU: {row.sentence_bleu:.2f}")
    print("EN: ", row.source)
    print("REF:", row.reference)
    print("MT: ", row.prediction)
```

**Interpret sentence scores cautiously.** Effective-order scoring is enabled only for these short-sentence diagnostics. Do not average the sentence scores to report corpus BLEU. A low score may indicate an error, but a fluent, faithful paraphrase may also overlap poorly with the single reference. Inspect meaning, grammar, negation, and omissions directly.

**Baseline scope.** A score on 20 teaching examples is unstable and cannot establish general translation quality. For later comparisons, use the same sources, references, decoding settings, and BLEU signature. Use the same hardware and timing procedure for speed comparisons. A stronger evaluation would use a larger, representative held-out English-French set and human review; overlap with the model training data has not been checked for these practice sentences.

# Save your outputs and reflect

## 6. Export UTF-8 JSONL and verify the saved data

Save one JSON object per source sentence. Store the prediction as a string and preserve accents. The round-trip check confirms that the saved data can be loaded without changing the records.

```
records = [
    {"id": i, "source": src, "reference": ref,
     "prediction": pred, "sentence_bleu": float(score)}
    for i, (src, ref, pred, score) in enumerate(
        zip(sources, references, predictions, sentence_scores), start=1
    )
]
output_path = Path("mt_baseline.jsonl")
with output_path.open("w", encoding="utf-8") as f:
    for record in records:
        f.write(json.dumps(record, ensure_ascii=False) + "\n")
with output_path.open(encoding="utf-8") as f:
    loaded = [json.loads(line) for line in f]
assert loaded == records and len(loaded) == len(pairs)
assert all(isinstance(r["prediction"], str) for r in loaded)
display(pd.DataFrame(loaded).head())
print("Verified JSONL:", output_path.resolve())
```

**Expected result:** 20 JSONL records, a five-row preview, and a printed output path. Correct indentation and the literal newline escape **\n** are essential: each record must occupy its own line.

## 7. Write your reflection

In your final markdown cell, write a brief **3-5 sentence reflection** addressing all of the following:

1. Report your corpus BLEU, device, batch size, and amortized time per sentence.
2. Quote one low-scoring prediction alongside its reference. Decide whether it is a translation error or a valid alternative, and explain your judgment.
3. Describe one observed quality issue, such as an omission, incorrect word sense, negation error, or unnatural phrasing. If you find no clear error, state that and identify a limitation of your sample or evaluation.
4. Propose one next experiment and explain how you would compare it fairly with this baseline.

## What you will submit

Submit your **executed .ipynb notebook** with the printed checkpoint and decoding settings, timing results, corpus BLEU and signature, three reviewed low-scoring examples, JSONL preview and verification, and final reflection. Keep **mt_baseline.jsonl** for later comparison.

In Colab, download the notebook through **File → Download → Download .ipynb**. Download the JSONL from the Files panel before the runtime is reset. Restart the runtime and run every cell in order before submitting.

## Completion check

Your notebook should finish without errors. Confirm that every source has a French reference and a nonempty translated string, BLEU is computed from those strings, and the reloaded JSONL exactly matches the 20 saved records. There is no required minimum BLEU score: your task is to establish and interpret a reproducible baseline.

## Technical references

[1] Helsinki-NLP. opus-mt-en-fr, model card and Transformers usage. English-French checkpoint.
[2] Post, M. (2018). A Call for Clarity in Reporting BLEU Scores. SacreBLEU documentation: Python API, reference layout, and metric signatures. SacreBLEU documentation.